

Análisis exploratorio del fraud_dataset

Conclusiones para la presentación – Ejercicio 1

SIA TP3 – ITBA 1Q 2026

1. Contexto y dataset

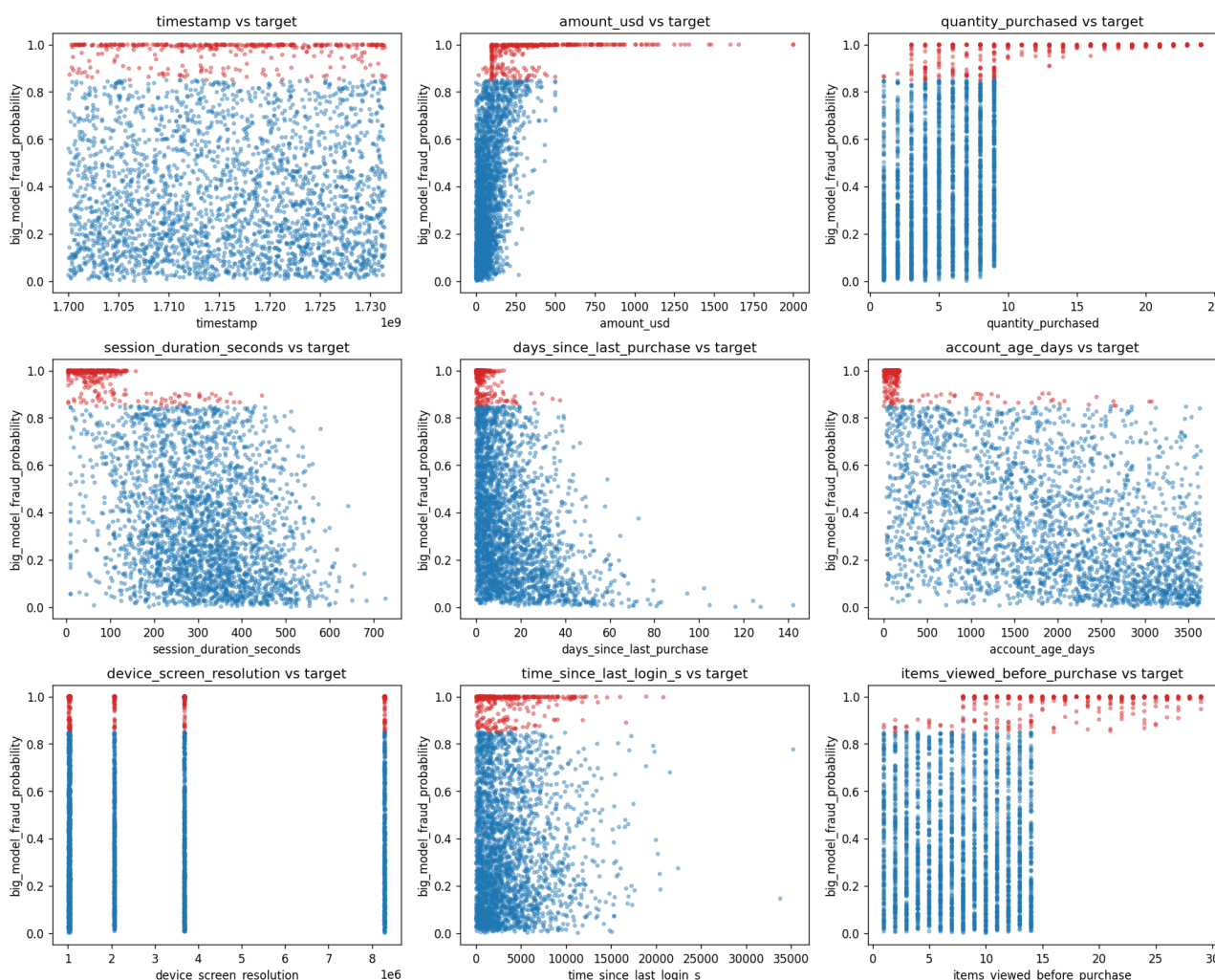
El `fraud_dataset.csv` contiene **7500 transacciones** con 9 features y dos columnas relacionadas al target:

- `big_model_fraud_probability` ($\in [0,1]$): salida del BigModel, es lo que el TinyModel debe *replicar* (knowledge distillation).
- `flagged_fraud` ($\in \{0,1\}$): ground truth, **solo para evaluación** (no se usa para entrenar).

Distribución de clases: 869 fraudes (11.59 %) vs 6631 legítimas (88.41 %). Existe **desbalance** relevante para la elección del umbral final y para las métricas que conviene reportar.

2. Vista general: scatter feature vs target

Para cada feature graficamos su relación con `big_model_fraud_probability`, coloreando por `flagged_fraud` (azul = legítimo, rojo = fraude real).



Observaciones a primera vista:

- En `amount_usd`, `quantity_purchased` e `items_viewed_before_purchase`, los puntos rojos (fraude) ocupan exclusivamente la parte derecha del eje X. Hay **umbrales duros** a partir de los cuales *todo* es fraude.
- `account_age_days` muestra dos modos casi disjuntos: el rojo se concentra a la izquierda, el azul a la derecha.
- `timestamp`, `device_screen_resolution` y `time_since_last_login_s` muestran nubes mezcladas sin estructura aparente: ambos colores se solapan en todo el rango.

3. Hallazgo principal: tres reglas determinísticas

Tres features tienen **umbrales que separan perfectamente** las clases en el dataset. Combinadas como OR:

$$\text{amount_usd} > 500 \vee \text{quantity_purchased} \geq 10 \vee \text{items_viewed_before_purchase} \geq 15$$

3.1. Aporte de cada regla individual

Regla	Matches	TP	FP
<code>amount_usd > 500</code>	199	199	0
<code>quantity_purchased ≥ 10</code>	524	524	0
<code>items_viewed_before_purchase ≥ 15</code>	490	490	0

Las matches se solapan: las tres reglas juntas marcan **695 filas únicas**.

3.2. Resultados de aplicar las reglas como predictor (simple_prediction)

		Predicción		Métrica	Valor
		1 (fraude)	0 (no fraude)		
Real	1 (fraude)	TP = 695	FN = 174	Accuracy	97.68 %
	0 (no fraude)	FP = 0	TN = 6631	Precision	100.00 %
				Recall	79.98 %

Regla descartada: `session_duration_seconds > 500` captura 1 TP y 350 FP. Rompería la precisión perfecta a cambio de un solo fraude más, por lo que se descarta.

El 80 % del fraude se etiqueta correctamente con tres if, sin un solo falso positivo. El 20 % restante (174 casos) es el verdadero desafío del perceptrón: son fraudes que respetan los tres umbrales y solo se detectan combinando features continuas.

4. Conclusiones por feature

Feature	Pearson r	Hallazgo principal
account_age_days	-0,585	Cuentas < 250 días contienen 87.8 % del fraude
quantity_purchased	+0,563	≥ 10 unidades: 100 % fraude (524 filas)
amount_usd	+0,557	> \$500: 100 % fraude (199 filas)
session_duration_seconds	-0,514	89.1 % del fraude tiene sesión < 150s
days_since_last_purchase	-0,404	84.5 % del fraude con < 5 días entre compras
items_viewed_before_purchase	+0,334	≥ 15 items: 100 % fraude (490 filas)
timestamp	+0,001	Sin señal lineal ni monótona
device_screen_resolution	+0,025	Sin señal lineal ni monótona
time_since_last_login_s	+0,002	Sin señal lineal ni monótona

Detalles relevantes:

- Las cuentas *nuevas* (< 250 días) son 87.8 % del fraude pero solo 5.7 % de las legítimas. Las cuentas *viejas* (> 1500 días) son 59.8 % de las legítimas y 0.9 % del fraude.
- En `days_since_last_purchase`, el patrón “compra reciente $\rightarrow$ probable fraude” es típico de uso de tarjeta robada (testeo + cargo grande en poco tiempo).
- Cuando hay fraude con sesión corta (< 150s), el BigModel está **muy confiado** (mediana de probabilidad = 0,997): señal fuerte que el TinyModel debería replicar.
- Las tres reglas duras son **discontinuidades**: en `quantity`, la tasa de fraude salta de $\sim 8\%$ en $q = 9$ a 100 % en $q = 10$. Lo mismo en `items`: $14 \rightarrow 15$.

5. Decisión: features a excluir del entrenamiento

Tres features se descartan del set de entrenamiento del perceptrón simple del Ej 1: `timestamp`, `device_screen_resolution` y `time_since_last_login_s`.

Validación con tres tests independientes:

Feature	Pearson	Spearman	Spread media-target / decil
timestamp	+0,001	+0,004	0.053
device_screen_resolution	+0,025	+0,029	0.051
time_since_last_login_s	+0,002	+0,001	0.021

- **Pearson** ≈ 0 : no hay relación *lineal*.
- **Spearman** ≈ 0 : tampoco hay relación *monótona*.
- **Media del target por decil** $\approx$ **constante** (alrededor del 0.42 global). Spread máximo de 5 %, dentro del ruido estadístico para ~ 750 muestras por decil.

Además, dos de las tres features tienen magnitudes problemáticas (`timestamp` en $1.7e9$, `device_screen_resolution` en $1e6$) que dominarían el cálculo del gradiente si no se normalizaran perfectamente.

Salvedad: Esta exclusión aplica al **perceptrón simple** del Ej 1. Para un MLP (Ej 2/3), dos features con $r \approx 0$ podrían aportar en *interacción* (e.g. $y = x_1 \cdot x_2$). Conviene reevaluar esta decisión al pasar a redes con capas ocultas.

6. Implicancias para el TinyModel

1. **Línea de base obligatoria:** cualquier perceptrón debe **al menos igualar** a `simple_prediction` ($\text{recall} \geq 80\%$, $\text{precision} \geq 100\%$ sobre los casos cubiertos por las reglas). Si no la iguala, hay un problema de capacidad o de normalización.
2. **Valor agregado del modelo:** se mide en cuántos de los **174 fraudes “sutiles”** logra recuperar.
3. **Lineal vs no-lineal:** las tres discontinuidades duras (saltos de $\sim 6\%$ a 100% en un solo entero) son geoméricamente imposibles de reproducir con una recta. Es donde el **perceptrón no-lineal con sigmoide** debe mostrar su ventaja respecto al lineal puro.
4. **Métricas:** con 88% de no-fraude, accuracy se infla sola. Reportar siempre *precision*, *recall* y curva ROC.
5. **Set de features final:** `account_age_days`, `quantity_purchased`, `amount_usd`, `session_duration_seconds`, `days_since_last_purchase`, `items_viewed_before_purchase` (6 features informativas).